

CI/CD Pipelines in Jenkins

Verfasser: Dario Cikojevic 4CHIT

Datum: 02.06.2026

1. Einführung

Diese Übung soll helfen die Funktionsweise und Einsatzmöglichkeiten von CI/CD Pipelines als DevOps Werkzeug zu verstehen. Als CI-Tool wird in diesem Beispiel Jenkins verwendet, da es sich dabei um eine der meistverbreitesten Ci/CD Plattformen handelt.

2. Projektbeschreibung

Das primäre Ziel dieses Projektes war der erfolgreiche Übergang von einer einfachen, sequenziellen "HelloWorld"-Pipeline zu einer vollautomatisierten, containerisierten Pipeline, welche den Kriterien einer vollständigen Aufgabenerfüllung entspricht.

Die Kernkomponenten umfassen:

- **Source & Checkout:** Dynamische Anbindung eines dedizierten GitHub-Repositories (`DoCC1502/Jenkins.git`) über das Jenkins SCM-Modul.
- **Isolierter Build & Unit-Test:** Kapselung einer Python-basierten Flask-API in ein Docker-Image, gefolgt von automatisierten Modultests (Unit-Tests) mittels `pytest` .
- **Dauerhaftes Hintergrund-Deployment:** Automatisches Stoppen veralteter Anwendungsversionen und unterbrechungsfreier Start des aktualisierten Docker-Containers im Hintergrundmodus (Detached) auf dem lokalen Host-System.
- **Automatisierung (Triggering):** Implementierung eines zyklischen Source Code Management (SCM) Pollings zur automatischen Pipeline-Ausführung unmittelbar nach einem Code-Commit.

3. Theorie

Zum Verständnis der gewählten Implementierungsstruktur sind folgende theoretische Grundlagen wesentlich:

3.1 Jenkins & Deklarative Pipelines

Jenkins wurde als zustandshaltender Docker-Container aufgesetzt. Es kommt eine *Deklarative Pipeline* via `Jenkinsfile` zum Einsatz. Diese bietet eine vordefinierte Struktur (Stages, Steps,

Post-Actions) und begünstigt das Prinzip von *Pipeline as Code*, wodurch die Infrastruktursteuerung versioniert im Git-Repository liegt.

3.2 Docker-outside-of-Docker (DooD)

Um aus einem laufenden Jenkins-Container heraus weitere Docker-Images zu bauen und Container zu starten, wurde der Docker-Daemon des Host-Systems mittels Socket-Binding (`-v /var/run/docker.sock:/var/run/docker.sock`) in den Jenkins-Container durchgereicht. Jenkins fungiert hierbei lediglich als CLI-Client, während die Ausführungskontrolle und das finale Deployment direkt auf dem Betriebssystem des Laptops (Host) stattfinden.

3.3 Testautomatisierung & API-Validierung

Die Qualitätssicherung erfolgt zweistufig. Zuerst prüft ein **Unit-Test** die Kernlogik der Route isoliert während des Build-Vorgangs. Nach erfolgreichem Deployment verifiziert ein systemischer **Integrationstest** über ein automatisiertes `curl` -Signal den Netzwerkpfad und die korrekte HTTP-Antwort (Statuscode 200, JSON-Payload) der Live-API.

4. Arbeitsschritte

4.1 Infrastruktur & Container-Initialisierung

Die Bereitstellung der Jenkins-Instanz mit administrativem Root-Zugriff und Docker-Socket-Anbindung erfolgte über die Windows PowerShell mittels strukturierten Backticks (```):

PowerShell

```
docker run -u root -d `
  -p 8080:8080 `
  -p 50000:50000 `
  -v jenkins_home:/var/jenkins_home `
  -v /var/run/docker.sock:/var/run/docker.sock `
  --name mein-jenkins `
  jenkins/jenkins:latest
```

Nach dem Login via Administrator-Passwort wurde die Docker-CLI innerhalb des Containers nachinstalliert, um die Kommunikationsfähigkeit mit dem Host-Daemon herzustellen:

Bash

```
apt-get update && apt-get install -y apt-transport-https ca-certificates curl gnupg  
lsb-release  
curl -fsSL https://download.docker.com/linux/debian/gpg | gpg --dearmor -o  
/usr/share/keyrings/docker-archive-keyring.gpg  
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-  
archive-keyring.gpg] https://download.debian.org/linux/debian $(lsb_release -cs)  
stable" | tee /etc/apt/sources.list.d/docker.list > /dev/null  
apt-get update && apt-get install -y docker-ce-cli
```

4.2 Anwendungsarchitektur & Test-Definition

Die Applikation wurde modular aufgeteilt. In `src/hello.py` befindet sich die produktive Flask-API, während in `tests/test_hello.py` die Testsuite implementiert wurde:

Python

```
# src/hello.py  
from flask import Flask, jsonify  
app = Flask(__name__)  
  
@app.route('/api/hello', methods=['GET'])  
def hello():  
    return jsonify({"counter": 31, "message": "Hello Spencer", "status": "success"})  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5556)
```

Das zugehörige `Dockerfile` zur standardisierten Kapselung der Laufzeitumgebung wurde im Root-Verzeichnis platziert:

Dockerfile

```
FROM python:3.11-slim  
WORKDIR /app  
COPY . /app  
RUN pip install flask pytest  
EXPOSE 5556
```

4.3 Pipeline-Konstruktion (Jenkinsfile)

Zur Eliminierung redundanter Ausführungen und zur Vermeidung blockierender unendlicher Schleifen (wie z.B. `sleep infinity`), wurde ein performantes, deterministisches `Jenkinsfile` entwickelt:

Groovy

```
pipeline {
    agent any
    environment {
        APP_PORT = '5556'
    }
    stages {
        stage('Checkout') {
            steps {
                cleanWs()
                checkout scm
            }
        }
        stage('Docker Build & Unit-Test') {
            steps {
                sh 'docker build -t mein-spencer-app .'
                sh 'docker run --rm mein-spencer-app python -m pytest
tests/test_hello.py -v'
            }
        }
        stage('Deployment auf lokalem Notebook') {
            steps {
                sh 'docker stop spencer-app-running || true'
                sh 'docker rm spencer-app-running || true'
                sh "docker run -d -p ${APP_PORT}:${APP_PORT} --name spencer-app-
running mein-spencer-app python src/hello.py"
            }
        }
        stage('Integrationstest (API-Check)') {
            steps {
                sleep 3
                sh "curl http://localhost:${APP_PORT}/api/hello"
            }
        }
    }
}
```

Fehlerbehebung (Refactoring): Ursprünglich kam es durch statische Verweise im alten Skript zur parallelen Initiierung von zwei Git-Repositories. Durch den Umstieg auf die native Jenkins-Variable `checkout scm` wurde sichergestellt, dass ausschließlich das eigene Repository (`DoCC1502/Jenkins.git`) verarbeitet wird.

4.4 Automatischer Commit-Trigger

Um die Pipeline ohne externe Webhooks direkt bei lokalen Datei-Pushes zu starten, wurde im Jenkins-Interface unter den Job-Optionen das **Poll SCM**-Verfahren aktiviert. Die Definition lautet:

- **Schedule:** `* * * * *` (Prüfung auf Code-Änderungen jede Minute).

5. Zusammenfassung

Im Rahmen dieser Laborübung wurden sämtliche Anforderungen der Ausbaustufen 6.1 und 6.2 vollständig erfüllt. Die Jenkins-Instanz läuft fehlerfrei und interagiert transparent mit dem Docker-Daemon des Host-Notebooks.

Durch das Auflösen des fehlerhaften Doppel-Checkouts und die Ersetzung der blockierenden Hintergrundprozess-Struktur (`sleep infinity`) durch ein sauberes, detached Docker-Hintergrund-Deployment (`docker run -d`) wurde eine professionelle CI/CD-Pipeline realisiert. Bei einem Push auf GitHub startet die Pipeline autonom, baut das Artefakt, führt die Unit-Tests aus und aktualisiert die produktive Schnittstelle auf Port 5556 unterbrechungsfrei mit einem finalen Status-Ergebnis von `Finished: SUCCESS` .